# Visualising Developer Interactions in Code Reviews

Daniel Bee
Royal Holloway, University of London
Egham, UK
Daniel.Bee.2022@live.rhul.ac.uk

DongGyun Han
Royal Holloway, University of London
Egham, UK
DongGyun.Han@rhul.ac.uk

## Abstract

Code review is an auditing process in which all changes are reviewed by someone other than the author of the changes. Because of its diverse benefits, as demonstrated by multiple studies, it has been widely adopted in both industry and open source projects. Various studies have proposed techniques to recommend reviewers who can audit changes, but there is limited support for comprehending existing review pairs (i.e., pairs of change authors and reviewers). Without this understanding, development teams may struggle to distribute workloads effectively. For example, a development team might overlook a concentrated workload on a specific developer, potentially slowing down the overall development process. On the other hand, some developers may submit code review requests (pull requests), but fail to receive timely attention from their colleagues.

To help developers understand the existing review pairs, we propose CRV (Code Review Visualiser). CRV visualises developers interactions during code reviews by quantifying their activities in a graph representation. In the graph, each node represents an individual developer while the edges denote the relationships based on discussions within the same code review requests. The size of a node indicates the total number of comments a developer has written in code review requests. The thickness of an edge between two nodes represents the number of comments that two developers authored within the same code review requests. Both nodes and edges leverage a colour scheme to represent the regularity of interactions. CRV also provides time window control, allowing developers to explore their code review activities across different time frames. We expect that CRV can help developers improve their code review practices by providing a better understanding of their current practices.

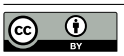
## CCS Concepts

• **Software and its engineering → Programming teams**.

## Keywords

code review, visualisation, developer interactions

**ACM Reference Format:**
Daniel Bee and DongGyun Han. 2025. Visualising Developer Interactions in Code Reviews. In *33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), June 23–28, 2025, Trondheim, Norway*. ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3696630.3728583




*FSE Companion '25, Trondheim, Norway*

ACM ISBN 979-8-4007-1276-0/2025/06
https://doi.org/10.1145/3696630.3728583

## 1 Introduction

Code review is a practice that all code changes are reviewed by one or more developers other than the change author themselves. For example, previous studies have shown that it is useful for sharing knowledge and development context within a development team. In addition, code review is helpful for improving software quality and design quality [6, 7]. Because of the diverse benefits, the practice has been widely adopted in both open and proprietary software projects. GitHub's pull request is a representative example of code review.

One of the most challenging parts of code review is finding suitable peers to review code changes. To mitigate the issue, many studies have proposed reviewer recommendation techniques [8, 10]. These techniques recommend a group of developers for a given code review request. However, these techniques are often limited to providing guidance for individual code review requests rather than providing project-wide guidance.

In this study, we propose a tool, CRV (Code Review Visualiser), that visualises developer interaction during pull requests on GitHub. Specifically, CRV visualises how frequently a developer participates in code reviews and how often pairs of developers interact within the same code review. Based on a given project, our technique provides a visualisation of developer activity within a selected timeframe. We expect that the visualisation will help software development teams understand the performance of their code review practices over time. It can potentially identify pairs of developers who are struggling to work together (e.g., those who rarely communicate). We also expect the visualisation to assist in planning effective developer pairings. For example, teams might group developers who actively communicate each other to boost development speed. Alternatively, they may pair developers who have rarely worked together to foster fresh synergy from new combinations.

CRV consists of three components: crawler, migrator, and visualiser. Since the visualisation requires multiple pull requests to comprehend developer interactions, requesting such a large amount of data from GitHub could be burdensome and slow. To avoid generating excessive traffic and to respond to user requests quickly, the crawler component retrieves pull request data from GitHub using the GitHub GraphQL API[1]. Once the crawling is done, the migrator component processes the crawled data and stores it in a database. Finally, the visualiser component visualises how developers interact with each other during code reviews.

We expect CRV can help developers improve their code review practices. For example, a development team can encourage a pair of developers to work together if they have collaborated on many code review requests. Alternatively, the team can consider changing code review pairs to bring in a fresh perspective.

[1] https://docs.github.com/en/graphql

## 2 Developer Interactions

Listing 1 shows the algorithm to extract interactions from pull requests for the visualisation. We represent the interactions as a graph. Each node represents a developer and its size denotes the number of comments the developer has written. Each edge between two nodes indicates the communication between two developers (PR author and reviewer) within PRs, and its thickness reflects the frequency of their interactions. Simply counting the frequency, however, the visualisation might be biased. For example, if a developer writes 100 comments on a single day, and they may appear as the most interactive developer in the visualisation. Based on the frequency, it is true that the developer is most active, but this does not necessarily reflect regularity. To address the issue, we use a colour scheme to represent the number of active days for each node and edge.

```
prs = [] # Pull requests must be provided
timeframe = (start_date, end_date)

nodes = {} # A list of nodes representing developers
links = {} # A list of links representing interactions

for pr in prs:
  if timeframe is given and pr is not within the
     timeframe:
    continue

  for comment in pr.comments:
    if comment.author in nodes:
      node = nodes[comment.author]
    else:
      node = Node(comment.author)
      nodes.append(node)
    node.count += 1
    node.addDate(comment.date) # for regularity

    author_pair = [pr.author, comment.author]
    author_pair.sort()
    if author_pair in links:
      link = links[author_pair]
    else:
      link = Link(author_pair)
      links.append(link)

    link.count += 1
    link.addDate(comment.date) # for regularity

for node in nodes: # Calculate regularity of nodes
  node.regularity = node.getNumDays() / timeframe.days
for link in links: # Calculate regularity of links
  link.regularity = link.getNumDays() / timeframe.days
```

**Listing 1: Algorithm to extract interactions**

When PRs are given, the algorithm iterates all PRs to extract information for the visualisation. If a timeframe is given to the algorithm, it only uses the PRs created within the timeframe. Otherwise, the algorithm leverages the given PRs. For the PRs included in the scope of the visualisation (i.e., no timeframe is given or PRs created within the timeframe), the algorithm iterates all comments in them. Using the comment author information, we increase the count of the corresponding node by one. In addition, we construct a list that contains both PR and comment authors, then increase the count by one. To avoid unnecessary duplicates (e.g., $\{JohnDoe, JaneDoe\}$ and $\{JaneDoe, JohnDoe\}$), we sort the list before increasing the count. The count of author pairs (i.e., `links`) will be used to control the thickness between developer nodes in the visualisation. Both node and link maintain the comment date information to visualise the regularity of interactions. By calculating the ratio of unique comment dates to the total duration of given timeframe in days, a normalised value is derived to represent regularity.

## 3 Visualisation Framework

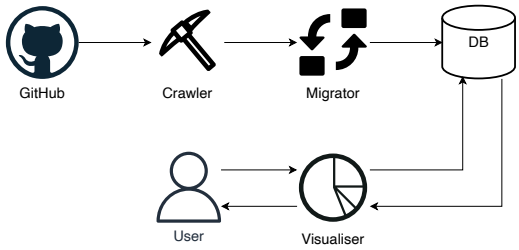



**Figure 1: Overall framework**

The visualisation framework consists of three components as shown in Figure 1: Crawler, Migrator, and Visualiser. The crawler retrieves PRs and comments from GitHub. Then, migrator parse the data and store it to a database. Finally, a user can see a visualisation of a project via the web based visualiser. Since the framework is independent from the main GitHub repository and optimises its data for visualisation, it can effectively handle large repositories.

In this section, we explain the role of each component and implementation details. The implementation details can be found in our replication package[2].

### 3.1 Crawler

**Table 1: Data Properties**

| Property | Description |
|---|---|
| `number` | Numeric id for pull request |
| `author.login` | Unique identifiers of developers |
| `author.avatarUrl` | URL of developer profile picture |
| `author.__typename` | User type (either human user or bot) |
| `createdAt` | Timestamp of PR/comment |

To visualise developer interactions, we need a large amount of pull request data. It will introduce huge traffics to code review platform if we request data whenever we need them. Such huge traffics potentially slow down the code review platform, so we do not query directly to the code review platform. Instead, we implemented a crawler that can crawl the latest code review data. The crawler leverages GitHub GraphQL API and implemented in Python. We constructed a query that retrieves `author` and `createdAt` properties of PRs and PR comments. Table 1 lists the properties that we crawled.

### 3.2 Migrator

GraphQL is a powerful approach handling large scale requests, but using GitHub GraphQL API directly for visualisation has three outstanding issues. First, GraphQL has additional types of nodes to optimise queries. For example, both PR and comment nodes should be surrounded by the `edges` and `node(s)` nodes which introduce

[2]https://github.com/DanJBee/CodeReviewVisualisation

unnecessary depth navigation. Second, GraphQL supports only limited conditional query. One of our visualiser's features supports visualising developer interactions within a specific time window. However, GraphQL does not support query that returns PRs that were authored within a specific time range. Last, traversing all PRs take time and require high computation power on the PR platform.

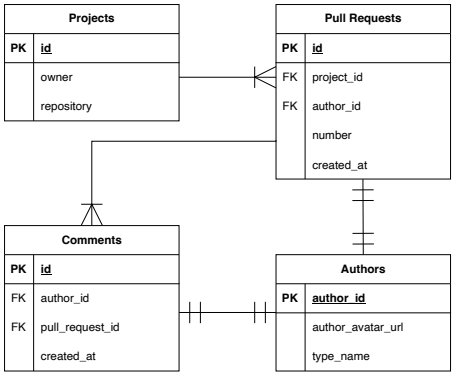



**Figure 2: Entity Relationship Diagram**

To mitigate the issues, we implemented a migrator that migrates the crawled API responses to a relational database instance. Figure 2 presents an entity relationship diagram of our database schema. The schema represents PRs and comments with the minimum subset of data that is required for visualisation. The migrator was also written in Python, and we evaluated it with a MariaDB instance[3]. During migration, we excluded all users whose type name (i.e., `author.__typename`) is "Bot" to avoid including automatically generated comments.

## 3.3 Visualiser

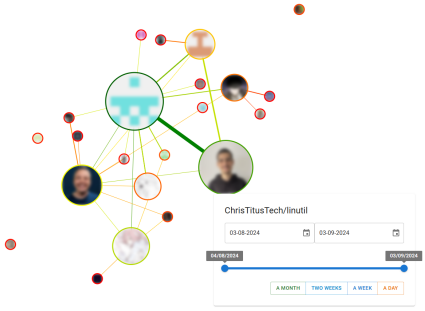



**Figure 3: An example of visualisation. We blurred the actual profile photos to protect developers' privacy**

The visualiser is responsible for visualising developer interactions. Users can select a specific project and time window to visualise developer interactions. For example, a user can request the visualiser to visualise developer interactions in the TypeScript project for a week from a specific date.

We implemented the visualiser as a web service that consists of two main components: backend and frontend. The backend component was implemented by using Java and Spring Framework. It retrieves data from the database upon request and run the algorithm described in Section 2. The results are serialised into a JSON string and return to the frontend. The frontend component is implemented by using TypeScript and React Framework. Since developer interactions can be visualised as a graph, we adopted the Force-directed graph component[4] in D3.

Figure 3 shows an example of the visualisation. Based on the algorithm presented in Section 2, the size of each node represents the number of comments written by each developer with their avatar image on GitHub. Please note that we blurred images on the paper, but real implementation shows clear images. The thickness of edges between nodes denote the number of comments shared by the two developers within the same PRs. To handle extremely large or small cases, we use normalised values for both nodes and edges. The colours of the nodes and edges dictate the regularity of each developer within the given timeframe. For instance, if a developer contributed every day of the chosen timeframe then they are coloured green, whereas if they only contributed once they are in red. Any values in between are gradient values between red, yellow, and green.

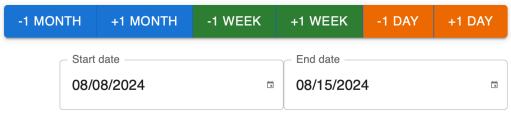



**Figure 4: Date Controller**

On the right bottom of the frontend, we put a date controller as shown in Figure 4. The user can select specific date by clicking the date field. Once they select a pair of dates, they can easily update the date scope by clicking the buttons above. Whenever the dates updated, the graph visualisation is also dynamically updated.

## 4 Potential Use Cases

We believe that our system has several use cases and real-world applications and will make it easier to make data-driven decisions for a project. For instance, it is paramount in identifying inequitable workloads in a development team. Our visualisation helps developers identify members of the team who are reviewing or being reviewed more than others [5].

Specifically, it can be a great help to open source projects. In Figure 3, we can observe several independent nodes that are separated from the graph. These independent nodes in the visualisation indicate that they have contributed to the project, but no one has interacted with them during observed the time period. In our preliminary experiments with multiple projects, we repeatedly observed the independent developers who could not connect to the main graph network regardless of the length of timeframe. This likely indicates a potential scenario in which several newcomers joined the project, but no one welcomed them, leading them left. If the project developers recognise the issue via our visualisation, they could encourage the newcomers to stay involved in the project.

By using the visualisation, we expect developers can plan their code review practices in a better way. If the nodes show a star topology (i.e., a developer participated in all discussions, and there

[3]https://mariadb.com

[4]https://observablehq.com/@d3/force-directed-graph

are no interactions between other peer developers), the team can consider distributing the workload of the developer who located at the centre of the discussion and fostering discussions between peers. In this way, the development team can plan to manage potential risks if the core developer leaves the project.

A development team can consider refreshing their code review practices based on the visualisation. If a pair of developers has been working together for a long time, the team could consider pairing them with a new developer who has worked on the same project, but rarely collaborates with them. These new pairings may help foster creativity among developers.

In addition, the visualisation could help enhance developer recognition and motivation and reduce resentment between members of a team. The size of each developer's contributions (i.e., the size of the nodes in the graph) are a useful quantitative measure of the work balance within the team [4]. Developers are encouraged to knowledge share more frequently as a result of information silos discovered through analysis of graph nodes. Node edge thicknesses help supplement how much intelligence is being shared between team members. Moreover, the code review process can be optimised more thoroughly and the necessary improvements in team dynamics can be made[2].

## 5 Related Work

There have been many studies aimed at helping developers find code reviewers. Thongtanunam et al. [8] argued that the developers working on large systems struggle to find appropriate reviewers. They assume that a developer who has previously changed the same file could be a good candidate as a reviewer, as they already understand the details of the file. Based on the assumption, they proposed a reviewer recommendation technique called RevFinder. Zanzani et al. [11] also proposed an automated reviewer recommendation approach that leverages previously completed reviews. These approaches are useful for developers to find potential reviewers. However, they do not provide insight of the current review practices. We believe our approach can provide additional information regarding the reviewer recommendation results.

In addition, many empirical studies have been conducted to understand developer interactions during software development. Hong et al. [3] investigated developer social network (DSN) of Mozilla projects and its evolution. They also compared the structure of DSN with general social networks (GSNs) such as Facebook and Twitter. Thung et al. [9] empirically studied social network structure on GitHub. They constructed developer-developer and project-project relationship graphs and identified influential developers and projects by using PageRank. However, their analysis was conducted at a project level, not at the code review level, and they did not provide a tool to visualise the graphs. Bock et al. [1] proposed a method that automatically identifies core developers by using role permissions of privileged events triggered in GitHub issues and pull requests.

## 6 Conclusion

In this paper, we propose CRV, a web based tool that visualises developer interactions during code reviews. CRV visualises developer activities and relationships during code reviews in a graph representation. Each node in the graph corresponds to an individual developer, with the node size indicating the frequency of their contributions to code review discussions. Each edge represents a collaborative relationship between a pair of developers with the edge thickness denoting the frequency of their collaboration within the same PRs. To provide visualisation without potential side effects (e.g., excessive traffic to the code review platform), we implemented three components: crawler, migrator and visualiser. The crawler leverages GitHub GraphQL API to retrieve PR data, including code review discussions, from GitHub. Then, the migrator parses the retrieved data and stores it in a database for flexible and faster querying. Finally, the visualiser fetches the data from the database and generates a visualisation based on the specified time frame.

As future work, we plan to conduct an empirical study to investigate the impact of the code review visualisation in practice. We anticipate our approach will lead to a better understanding of code review practice. With this improved understanding, developers may be able to distribute heavy workloads more effectively among their peers or facilitate the onboarding of previously unnoticed newcomers to their projects.

## 7 Replication Package and Screencast

The source code of CRV is available at https://github.com/DanJBee/CodeReviewVisualisation and a video demonstration of CRV is available at https://www.youtube.com/watch?v=L8YVGmvRkP4.

## References


[1] Thomas Bock, Nils Alznauer, Mitchell Joblin, and Sven Apel. 2023. Automatic Core-Developer Identification on GitHub: A Validation Study. *ACM Transactions on Software Engineering Methodology* (2023).

[2] Roy Gelbard and Abraham Carmeli. 2009. The interactive effect of team dynamics and organizational support on ICT project success. *International Journal of Project Management* (2009).

[3] Qiaona Hong, Sunghun Kim, S.C. Cheung, and Christian Bird. 2011. Understanding a developer social network and its evolution. In *2011 27th IEEE International Conference on Software Maintenance (ICSM)*.

[4] John W Koning Jr. 1993. Three other R's: Recognition, reward and resentment. *Research-Technology Management* 36, 4 (1993), 19–29.

[5] Mohsin Malik, Shagufta Sarwar, and Stuart Orr. 2021. Agile practices and performance: Examining the role of psychological empowerment. *International Journal of Project Management* (2021).

[6] Shane McIntosh, Yasutaka Kamei, Bram Adams, and Ahmed E. Hassan. 2014. The Impact of Code Review Coverage and Code Review Participation on Software Quality: A Case Study of the Qt, VTK, and ITK Projects. In *Proc. of the Working Conference on Mining Software Repositories (MSR)*.

[7] Rodrigo Morales, Shane McIntosh, and Foutse Khomh. 2015. Do Code Review Practices Impact Design Quality? A Case Study of the Qt, VTK, and ITK Projects. In *Proc. of the International Conference on Software Analysis, Evolution, and Reengineering (SANER)*.

[8] Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Raula Gaikovina Kula, Norihiro Yoshida, Hajimu Iida, and Ken-ichi Matsumoto. 2015. Who should review my code? A file location-based code-reviewer recommendation approach for Modern Code Review. In *IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*.

[9] Ferdian Thung, Tegawendé F. Bissyandé, David Lo, and Lingxiao Jiang. 2013. Network Structure of Social Coding in GitHub. In *2013 17th European Conference on Software Maintenance and Reengineering*.

[10] Yue Yu, Huaimin Wang, Gang Yin, and Tao Wang. 2016. Reviewer recommendation for pull-requests in GitHub: What can we learn from code review and bug assignment? *Information and Software Technology* (2016).

[11] Motahareh Bahrami Zanjani, Huzefa Kagdi, and Christian Bird. 2016. Automatically Recommending Peer Reviewers in Modern Code Review. *IEEE Transactions on Software Engineering* (2016).